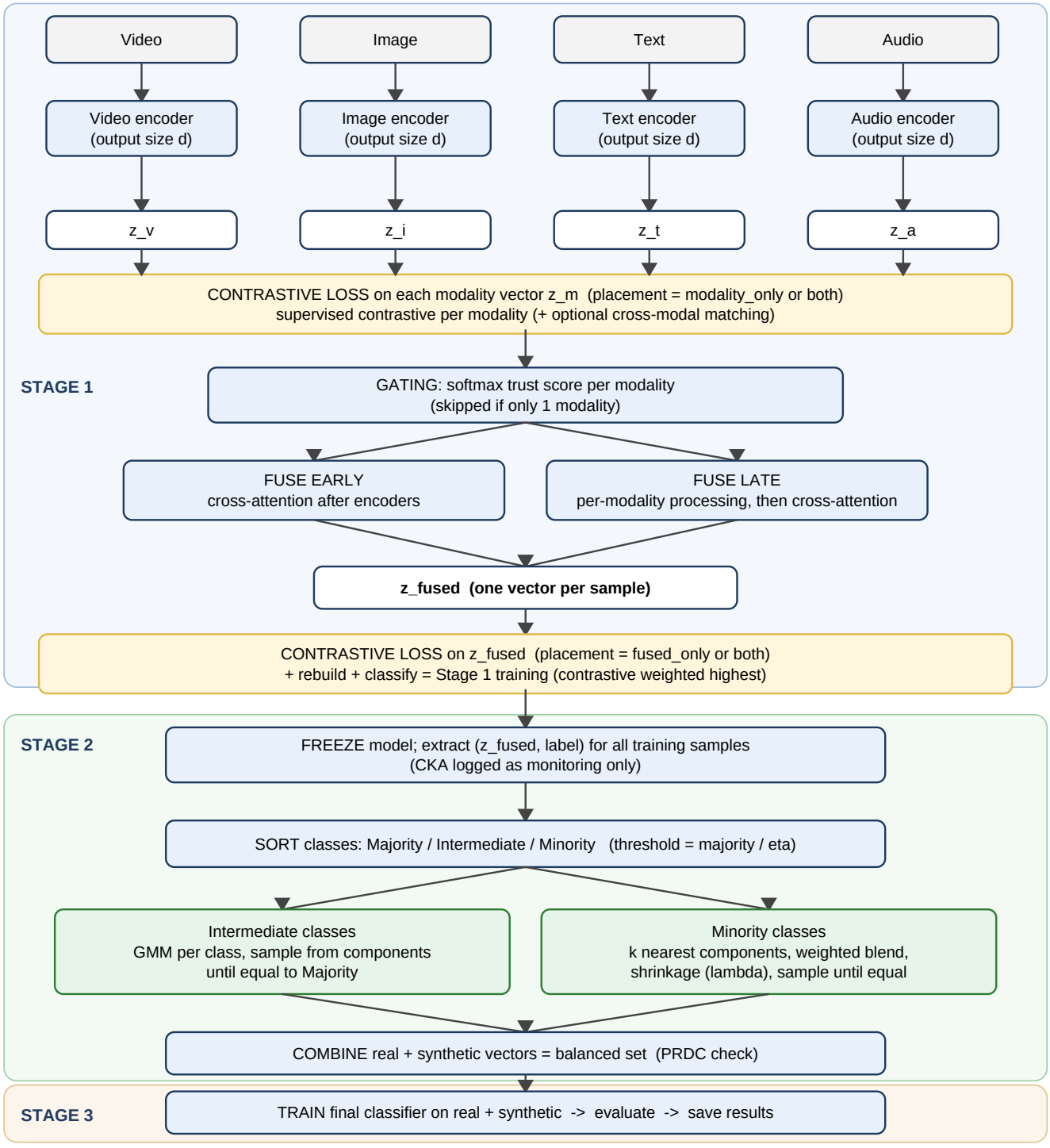
Multimodal RCS — Implementation Spec (Three Stages)

Condensed build instructions for the coding agent, organised as Stage 1, Stage 2 and Stage 3

Hard rules (all stages): (1) Dataset-agnostic: no dataset-specific code. (2) Enabled modalities come from config; any subset of video / image / text / audio. (3) Use the authors' RCS code exactly; never re-implement its math. (4) **No hyperparameter is hard-coded**; all are config values or Optuna-tuned. (5) Contrastive loss placement is a config switch (modality_only / fused_only / both) and contrastive gets the largest loss weight. (6) Only samples with all enabled modalities are used (project handles class imbalance, not missing modalities). (7) The test set is never used for tuning.

Overview



Orange = contrastive loss locations. Green = RCS (authors' code). Everything bounded by enabled modalities and Optuna-chosen values.

Stage	Purpose	Input	Output	Optuna
1. Representation learning	Learn a fused vector z_fused in which all classes are well separated	raw modalities + labels	frozen model; z_fused vectors for train / val / test	Stage A (high cost)
2. RCS balancing	Generate synthetic z_fused vectors for Intermediate and Minority classes	train z_fused + labels	balanced train set (real + synthetic)	Stage B (low cost)
3. Classification and evaluation	Train final classifier, compare variants and baselines, save results	balanced train set; val/test z_fused	metrics, plots, results folder	Stage B (with Stage 2)

Common Setup (before Stage 1)

- Each dataset = a manifest (CSV/JSON): `sample_id, label, split, <path per modality>`, plus a small adapter per dataset. Pipeline reads only the manifest.

```
dataset_name: CREMA-D
manifest: path/to/manifest.csv
modalities: [video, audio]          # any subset
fusion_timing: [early, late]
contrastive_placement: [modality_only, fused_only, both]   # run all 3 as ablation
create_artificial_imbalance: false  # only if dataset is balanced; train split only
```

- Drop samples missing any enabled modality (log per-class counts before and after). Infer number of classes from labels.
- Validation/test sets balanced where possible; otherwise report per-class and macro metrics.

STAGE 1 — Representation Learning

Goal: all classes must be well separated in the fused vector z_fused. The contrastive loss is therefore the **main** loss, and it must be the **supervised** kind (uses labels).

1.1 Model

- **Encoder registry**, one per modality: image = CNN; audio = spectrogram + CNN; video = frame CNN + temporal pooling; text = transformer/embedding. All output size d. Build only enabled ones. Check each encoder alone first; if one modality does not separate classes, fusion cannot fix it.
- **Gating:** small network over all M vectors, softmax trust scores summing to 1, scale each vector. Skip if M = 1.
- **Fusion:** cross-attention; each modality queries the concatenation of the others, results pooled to z_fused. **Early** = right after encoders; **Late** = after per-modality extra processing. Run both as separate experiments.

1.2 Losses and contrastive placement

Loss	Uses labels?	Effect
Supervised contrastive (SupCon)	Yes	Pulls same-class samples together, pushes different classes apart. **This is the loss that separates classes.**
Cross-modal InfoNCE (optional)	No	Aligns modalities of the same sample. Does not separate classes by itself. Keep as a small optional term.
Rebuild (MSE)	No	One decoder per modality from z_fused (text: reconstruct embedding).
Classify (cross-entropy)	Yes	Classifier head on z_fused.

Placement	SupCon applied on	Expected effect
modality_only	each modality encoder output z_m (before gating/fusion), one projection head per modality	Each modality space becomes class-separated. No direct guarantee on z_fused.
fused_only	z_fused only (after fusion), one projection head	Directly targets the space of interest. Weak if an encoder is weak.

Placement	SupCon applied on	Expected effect
both	each z_m AND z_fused	Usually best separation; more weights to balance (use Optuna).

- Downstream stages use z_fused, never the projection outputs.

```
L_total = w_con_fused * SupCon(z_fused) + w_con_mod * SupCon(z_m, each modality)
        + w_cls * CE + w_rec * MSE + w_align * InfoNCE(optional)
```

- **Contrastive weights highest:** start w_con at 1.0 to 2.0 with w_rec and w_cls at or below 1.0. Optuna searches w_con over about 0.5 to 5.0 (log scale) and w_rec, w_cls over a lower range. Terms for a placement that is switched off are 0.
- Tune SupCon temperature (around 0.07, search 0.03 to 0.3). Optional warm-up of the weight over the first epochs.
- Use a **class-balanced batch sampler** or larger batch size so every class has same-class pairs in each batch.
- Log **CKA** between z_fused and each modality vector every epoch (monitoring only). Watch for gating collapse onto one modality.

1.3 Optuna Stage A (tunes this stage only)

- d; encoder depth/width; gating size; attention heads (must divide d), layers, dropout; contrastive weights (fused and per-modality), rec/cls/align weights; contrastive temperature; projection size; lr; batch size; weight decay; epochs.
- Objective: validation separation / Macro-F1 of z_fused (e.g. kNN or linear probe). TPE sampler + median pruner.

1.4 Exit of Stage 1

- Freeze encoders, gating and fusion. Extract (z_fused, label) for **train, validation and test** samples and save them to the features folder (no augmentation on val/test).
- **Check before Stage 2:** compute separation metrics on z_fused (kNN accuracy, linear probe, silhouette, t-SNE/UMAP) for each placement. If separation is poor, fix Stage 1 first (see weights, sampler, encoder strength) and do not proceed.

STAGE 2 — RCS Balancing in the Fused Space

Goal: balance the classes using only the saved **training** z_fused vectors. Use the authors' RCS code exactly; only numeric parameters come from config / Optuna.

- **Sort classes:** threshold = majority size ÷ **eta** into Majority / Intermediate / Minority.
- **GMM per class:** components = max(1, round(class size ÷ smallest class size)); store mean, covariance, sample count, weight per component.
- **Intermediate classes:** split the needed count evenly over components; sample until equal to majority size.
- **Minority classes:** for each real point find the **k** nearest components of Majority/Intermediate GMMs; weight each by 1 ÷ its sample count; blend means/covariances with the point; apply shrinkage **(1 - lambda) x cov + lambda x I**; sample until equal to majority size.
- Wrap the authors' code so **eta, k, lambda and every other exposed argument** are passed in from config/Optuna.
- **Check:** compute **PRDC** (precision, recall, density, coverage) of synthetic vs real minority vectors to detect clumped samples.
- **Output:** balanced training set = real + synthetic vectors, saved to the features folder. Validation and test vectors are never augmented.

STAGE 3 — Final Classifier and Evaluation

Goal: train the classifier on the balanced set and measure honestly whether the method helps the rare classes.

- **Classifier:** MLP on real + synthetic vectors, cross-entropy, AdamW. Layers, nodes per layer, dropout, activation, lr, weight decay, epochs = Optuna.

- **Optuna Stage B** (Stages 2 and 3 together, on cached z_fused, cheap): eta, k, lambda, GMM options, classifier parameters. Objective: validation Macro-F1 or Balanced Accuracy (optional second objective: minority recall). **Never tune on the test set.**
- **Studies:** one SQLite study per dataset + modality set + fusion timing + placement; conditional parameters for enabled modalities only.
- **Final:** retrain with best params over several seeds; report test metrics (mean, std) from that run only.

Variants and baselines

Version	Fusion	RCS
A	Early cross-attention	No
B	Late cross-attention	No
C	Early cross-attention	Yes (main method)
D	Late cross-attention	Yes

- Every variant is run for each contrastive placement (modality_only, fused_only, both). Baselines: no correction, random oversampling, SMOTE-style interpolation on z_fused.

Metrics

- Balanced Accuracy, Macro F1, MCC, G-Mean, minority-class recall, per-class F1 (not plain accuracy).
- Separation of z_fused (kNN, linear probe, silhouette), PRDC, CKA, and t-SNE/UMAP before vs after augmentation. Compare across the 3 placements.

Results Layout (all stages)

```
results/<dataset>/<early | late>/<modality_only | fused_only | both>/<modalities>/
e.g. results/CREMA-D/early/both/video_audio/
    stage1_representation/  checkpoints/, features/ (z_fused train/val/test),
                            separation_metrics.csv, CKA log, plots/
    stage2_rcs/             gmm_params, synthetic vectors, PRDC.csv, plots/
    stage3_classifier/      model, metrics.csv, per-class F1, plots/
    best_params.json, config_snapshot.yaml, optuna/ (study.db, trials)
results/<dataset>/baselines/   (no correction, oversampling, SMOTE)
```

- Each dataset has its own folder; below it: fusion timing, then contrastive placement, then the modality set (keeps runs with different enabled modalities from overwriting each other).